LogicWhileLiRE: Go-Live & Optimization Playbook (Plan A, under Rs 5,000)

How to use this: Do the phases in order. In each phase, run the Antigravity prompt, check the result yourself, then commit. Never move on with a failing build.

Rules for working with Antigravity

1. Work on a new branch: `git checkout -b prod/go-live`. Commit after every phase.
2. One phase = one prompt. Big mixed prompts produce sloppy changes.
3. Always end prompts with: "Run `npm run build` in server and client and `npm test`. Fix every failure. Summarize changed files."
4. Never paste real secrets into a prompt. Put them only in `.env` files and the Render/Netlify dashboards.
5. If Antigravity touches something you did not ask for, say: "Revert unrelated changes, keep only what I asked."

Phase 0: Safety net (15 min)

- Push the repo to a **private** GitHub repository.
- Confirm `.env` is in `.gitignore`.

Prompt 0

```
Audit the repo for launch readiness. Do NOT change code yet.
1. Check .gitignore covers all .env files, node_modules, build output.
2. Search the whole repo for hardcoded secrets, passwords, API keys, localhost URLs.
3. Create server/.env.example and client/.env.example listing every env var the code reads, with plac
4. Give me a table: variable, used where, required in production yes/no.
```

Phase 1: Security hardening (before anything public)

Prompt 1

```
Harden the app for public production. Keep all existing features working.
- Remove or disable the seeded demo accounts (admin/admin123, suresh@, priya@) when NODE_ENV=producti
- Add helmet with sensible CSP, strict CORS allowing only the CLIENT_URL env var, and express-rate-li
- Validate every request body with zod (or the existing validator) on all auth, exam and admin routes
- Make sure admin routes check the role server-side, not just in the UI.
- Make sure correct answers are never sent to the client before submission.
- Cookies: httpOnly, secure, sameSite set correctly for Netlify frontend + Render backend on a custom
- DEMO_MODE must be impossible to enable when NODE_ENV=production.
Add tests for each item. Run builds and npm test, fix failures, summarize.
```

Phase 2: Real database on Supabase

Manual steps

Manual steps

1. supabase.com → New project (choose the region nearest India, e.g. Mumbai). Save the DB password.
2. Copy Project URL and service_role key. Put them only in `server/.env` and later in Render.

Prompt 2

```
Prepare the production Supabase (PostgreSQL) setup.
- Generate a single ordered SQL migration set in /supabase/migrations covering every table the code u
- Add indexes for every column used in WHERE/JOIN on hot paths (exam_code, hall_ticket, exam_id, stud
- Enable Row Level Security on all tables, deny-by-default, since only the server uses the service ro
- Write a production-safe seed script (colleges + one sample exam, no default passwords).
- Make the server fail fast with a clear message if Supabase is unreachable at boot.
- Test the same flows against real Supabase vs the mock store and list any behavior differences.
Give me exact steps to run the migrations in the Supabase SQL editor.
```

Phase 3: Email (response mail, auto-reply, OTP, admin@)

Manual steps

1. Buy the domain (GoDaddy / Namecheap / Hostinger India, about Rs 700 to 1,000 a year).
2. Zoho Mail free plan: add the domain, verify, create `admin@yourdomain`. Add the MX, SPF and DKIM records Zoho shows. (Free plan is web/app access; check Zoho's current limits.)
3. Sending mail from the app: create a Resend or Brevo account, add the domain, add their DKIM/SPF records, create an API key or SMTP credentials. Free tiers have daily caps, so check current limits before exam day.
4. Add a DMARC record: `v=DMARC1; p=none; rua=mailto:admin@yourdomain.`

Prompt 3

```
Build a production email layer.
- One mail service module using SMTP env vars (SMTP_HOST/PORT/USER/PASS, MAIL_FROM) so I can switch R
- Templates (branded HTML + plain text): OTP, password reset, welcome/registration confirmation, scor
- OTP: 6 digits from crypto, hashed in DB, 10 minute expiry, max 5 attempts, resend cooldown 60s.
- Queue with retry and exponential backoff; log failures to audit_logs; never block the HTTP response
- Respect daily provider limits: add a configurable per-day cap and a clear admin warning when near i
- Add a `npm run test-email` script that sends a test mail to an address I pass.
```

Phase 4: Deploy backend (Render) and frontend (Netlify)

Manual steps

1. Render → New → Blueprint → select the repo (uses `render.yaml`). Add every env var from `.env.example` in the dashboard. Set `NODE_ENV=production`, `DEMO_MODE=false`, `CLIENT_URL=https://yourdomain.`
2. Copy the Render URL, open `/api/health`, confirm OK.
3. Netlify → Import from Git → base dir `client`. Set the API/socket URL env var to the Render URL. Deploy.
4. UptimeRobot (free): HTTP monitor on `https://<render-url>/api/health`, every 5 minutes.

5. Domain: point `www` (and root) to Netlify, and `api.yourdomain` CNAME to Render. Add the custom domain in both dashboards and wait for HTTPS.

Prompt 4

```
Make deployment configs production-correct.
- Verify render.yaml and client/netlify.toml: build commands, Node version pin, SPA redirect to index
- Client must read the API and Socket.io URLs from env (VITE_*), no hardcoded localhost.
- Socket.io: configure CORS, websocket transport with polling fallback, heartbeat settings that survi
- Graceful shutdown (SIGTERM) so Render restarts do not drop active exam submissions.
- /api/ready must check the DB connection.
- Write DEPLOY.md with the exact click-by-click steps and env var table.
```

Phase 5: Performance and scale

Honest note: Render's free tier sleeps and has limited RAM/CPU. For a real exam day with 100+ students, use the paid starter plan for that week only (about Rs 600 a month). It fits inside your Rs 5,000.

Prompt 5

```
Optimize for 300 concurrent exam candidates on a small server (512MB-1GB RAM).
- Profile and fix N+1 queries and missing indexes; batch autosave writes (debounce on client, upsert
- Cache exam/question payloads in memory with TTL; avoid reloading the question bank per request.
- Compress responses (gzip/brotli), add ETag where safe.
- Limit Socket.io payload size and emit rates for the monitor wall; send diffs, not full snapshots.
- Client: check bundle with vite-bundle-visualizer, lazy-load admin routes, keep Three.js and MediaPi
- Add a k6 or autocannon script in /loadtest simulating login, fetch questions, 60 autosaves per cand
Report before/after numbers.
```

Phase 6: Reliability and monitoring

Prompt 6

```
Add production observability.
- Structured JSON logging (pino) with request IDs; never log passwords, tokens or answers.
- Sentry (free tier) for server and client with env-based DSN, source maps uploaded in build.
- Global error handler returning clean messages; React error boundaries with a friendly recovery scre
- Exam-safety: if the server restarts mid-exam, the client must keep working from the IndexedDB cache
- Admin page showing: last 24h errors, email failures, active sessions.
- Document a backup routine: Supabase daily backups plus a weekly `pg_dump` script.
```

Phase 7: Product polish and enhancement

Prompt 7

```
Polish UX without breaking the exam flow.
- Accessibility pass (labels, focus order, contrast, keyboard navigation) on login, enter-exam, instr
- Fully mobile-responsive layouts; test at 360px width.
- SEO + sharing on the landing page: title, meta description, OpenGraph image, sitemap.xml, robots.tx
```

```
- SEO + sharing on the landing page: title, meta description, Open Graph image, sitemap.xml, robots.tx
- Custom 404/500 pages, loading skeletons, empty states, clear error toasts.
- Consistent English/Telugu toggle across instructions and key candidate pages.
- PWA: manifest, icons, offline shell for the practice arena only.
```

Phase 8: Compliance and trust

Prompt 8

```
Add Privacy Policy, Terms of Use, and a proctoring consent screen shown before the exam (camera used
```

Phase 9: Final test and launch checklist

Prompt 9

```
Create an end-to-end test plan and run what can be automated (Playwright):
candidate: enter exam → instructions → exam → autosave → submit → result → leaderboard.
admin: login → create exam → import questions → exam code → monitor → publish results → export.
Also: wrong code, expired window, duplicate login, offline then reconnect, 3 violations auto-submit.
Output a pass/fail table and fix failures.
```

Your manual checklist

- ⬛ Real phone test on mobile data (not just Wi-Fi)
- ⬛ Send a real OTP and reset mail to Gmail, Outlook; check they do not land in spam (mail-tester.com)
- ⬛ Mock exam with 20 friends on different devices
- ⬛ Admin password changed, demo accounts gone
- ⬛ Supabase backup confirmed
- ⬛ UptimeRobot green; Render upgraded to paid for exam week
- ⬛ Share one rollback plan: previous Netlify deploy can be restored in one click

Phase 10: After launch

- Week 1: watch Sentry and Supabase logs daily; fix top 3 issues.
- Collect feedback from the first exam, then ask Antigravity: "Analyze these issues and propose a prioritized fix plan before coding."
- Later roadmap (from your gap analysis): TPO portal, granular admin roles, real code sandbox (Piston), Google Sheets sync, LMS.

Suggested budget

Item	Cost
Domain (1 year)	about Rs 1,000
[illegible]	[illegible]

Render paid, exam month	about Rs 600 to 1,200
Everything else (Netlify, Supabase, Zoho, Resend/Brevo, UptimeRobot, Sentry)	free
Buffer	about Rs 2,800